

Fainlly Backend

This is the Node.js (Express) backend for the Fainlly application, providing a REST API for user management, habit tracking, mood logging, daily notes, and AI-powered motivational messages.

Prerequisites

- Node.js (v18+)
- npm (or pnpm/yarn)
- MySQL or a compatible database (e.g., MariaDB)

Setup and Installation

1. **Clone the repository (or navigate to the project directory):** ````bash cd /path/to/fainlly_backend ````
2. **Install dependencies:** ````bash npm install ````
3. **Configure Environment Variables:** Create a file named ``.env`` in the root directory and copy the contents of ``.env.example`` into it. **You must fill in your actual database credentials and OpenAI API key.**
``.env.example`` content: `````

Database Configuration (MySQL/PostgreSQL)

`DB_HOST=localhost DB_USER=root DB_PASSWORD=password DB_NAME=fainlly_db`

Server Configuration

`PORT=3001`

AI Configuration (Using OpenAI as an example)

`OPENAI_API_KEY=YOUR_OPENAI_API_KEY_HERE AI_MODEL=gpt-4.1-mini ````

4. **Run the Server:** The server will automatically run the database migration (`schema.sql`) on startup if the tables do not exist.

```
```bash node index.js ```
```

The server will start on the port specified in your `.env` file (default: `3001`).

## Database Schema (Migration)

The database schema is defined in `schema.sql`. The server automatically executes this file on the first run to create the necessary tables: `users`, `habits`, `habit_days`, `moods`, and `notes`.

## API Endpoints

The API base URL is <http://localhost:3001/api>.

Endpoint	Method	Description	Required Data
<code>/users</code>	<code>POST</code>	Create user + select mode	<code>{ username, mode: 'boys' }</code>
<code>/habits</code>	<code>POST</code>	Add a new habit	<code>{ name, category }</code> (Requires <code>x-user-id</code> header)
<code>/habits/monthly</code>	<code>GET</code>	Fetch monthly habit table	Query params: <code>year</code> , <code>month</code> (Requires <code>x-user-id</code> header)
<code>/habits/toggle</code>	<code>PUT</code>	Update a habit day status	<code>{ habit_id, day: 'YYYY-MM-DD', status: true }</code>
<code>/moods</code>	<code>POST</code>	Save daily mood	<code>{ date: 'YYYY-MM-DD', mood_value: 1-5 }</code> (Requires <code>x-user-id</code> header)
<code>/moods</code>	<code>GET</code>	Fetch mood data	Query params: <code>start_date</code> , <code>end_date</code> (Optional) (Requires <code>x-user-id</code> header)
<code>/notes</code>	<code>POST</code>	Save daily note	<code>{ date: 'YYYY-MM-DD', content }</code>

			(Requires <code>x-user-id</code> header)
<code>/notes</code>	<code>GET</code>	Fetch daily note	Query param: <code>date</code> (Requires <code>x-user-id</code> header)
<code>/ai/motivational-message</code>	<code>GET</code>	Get AI-generated motivational message	(Requires <code>x-user-id</code> header)

**Note on Authentication:** For simplicity, all authenticated endpoints require a `x-user-id` header containing the user's ID.

## Frontend Integration

To connect the provided frontend to this backend:

1. Ensure the backend is running on `http://localhost:3001/`.
2. In the frontend project, you will need to update the API base URL configuration to point to `http://localhost:3001/api/`. This is typically done in an environment file or a constant file within the frontend source code.
3. The frontend must be configured to send the `x-user-id` header with every request to authenticated endpoints.

---

*Developed by Manus*